

DATA STRUCTURES AND ALGORITHMS II, 2023-2024

“CHAMPIONS LEAGUE BATTLE”

Marco de Paco, Víctor Ferré, Eloi Garcia
u231728, u232180, u231989
29/5/2024

PROJECT REPORT TEMPLATE

INTRODUCTION

This section should describe the context, problem definition, existing solutions you looked at for inspiration, your proposal and objectives met, and the structure of the report

The context of this project is to develop a turn-based combat RPG (Role-Playing Game) set in the world of football, specifically focusing on FC Barcelona's journey to win the 2015 Champions League. The game aims to provide an engaging experience where players can strategize their way through various scenarios and battles against iconic football opponents.

The proposal is to develop a turn-based role-playing game where players control one character from FC Barcelona and navigate through scenarios inspired by iconic European cities. Players will choose skills, engage in strategic combat with opponents, and make decisions that impact the outcome of the game. The game will combine elements of football gameplay with RPG mechanics to create an immersive experience. So, our main goal was to create an interactive and entertaining game experience that captures the essence of football while incorporating RPG elements like character customization, skill selection, and strategic combat.

This report is organized as follows: the next section explains the project's objectives and how they were achieved, including details on implementation and performance evaluation. Then, we discuss the developed solution, including system architecture, error handling, data model design, and data processing. We also cover ethical considerations to ensure academic integrity and responsible use of data. Finally, we list the references used in the project.

PROJECT OBJECTIVES

Mandatory objectives met

Our first objective was to involve the creation of the main character with essential attributes including name, HP, attack (ATK), defense (DEF), and a characteristic attack. This was achieved by defining the Character struct, which holds these attributes. Each attribute such as name, hp, atk, def, and characteristic_attack is assigned accordingly within the struct. The implementation for this objective is located from the beginning of the code until line 44. (2 hours)

The second goal was creating scenarios with decision-making options to influence the game's narrative and progression. Each scenario, defined by the Scenario struct, includes a name, description, and decision options. The Decision struct encompasses a question and two options, each with corresponding narratives before and after the decision. This implementation allows players to engage with the game's storyline and shape their experience through strategic choices. It took approximately 2 hours to develop this objective, involving the structuring of scenario data and decision-making mechanics. (4 hours)

Objective 4 involves implementing a basic turn-based battle system where the player and enemies engage in combat. The combat function manages battle mechanics, including attack, defense, and HP calculation based on skills and attributes. It utilizes variables such as player_choice, player_damage, enemy_choice, and enemy_damage to execute turn-based actions. The algorithm orchestrates the flow of combat, ensuring fairness and strategic depth. This objective required approximately 3 hours to develop, encompassing the design and implementation of combat mechanics within the game. (6 hours)

Another objective was to develop different types of enemies, enemies are represented by the data structure Enemy, which contains fields for the enemy's name, health (hp), attack (atk), defense (def), and skills (skills). Each enemy can have up to 4 different skills that they can employ during combat. These skills are defined as an array of structures of type Skill. Each skill has a name, description, type (whether it's a temporary modifier or a direct attack), duration (in the case of a temporary modifier), and modifiers that affect the player's statistics (attack, defense, or health). During combat, a skill is randomly selected from the enemy's repertoire to use against the player, and the damage inflicted on the player is calculated based on the selected skill and the enemy's and player's statistics. This process repeats until one of the two (the player or the enemy) runs out of health points. In summary, the Enemy data structure, along with an array of Skill structures, is used to model and manage enemies and their skills during combat. (2 hours)

The sixth objective was to implement the "Time Strike," which at a certain point in the game gave us the option to double the power of a selected skill. We have been able to implement it in such a way that the player can use this ability at the moment they believe is necessary to defeat the opponent. (1 hour)

Desirable objectives met

For these desirable objectives we were just able to do two of them. These two were the following:

Including the implementation of a dictionary structure to track the usage count of the player's abilities, where the keys are the names of the abilities and the values are integers that increment with each use. This functionality is integrated into the game's menu, allowing players to see how often each ability has been used. The dictionary is dynamically updated as abilities are used, ensuring real-time tracking and providing valuable feedback to the player about their gameplay strategy and ability utilization. (4 hours)

Including execution time analysis for at least two key functionalities that utilize data structures or algorithms studied in class, such as stacks, queues, dictionaries, search and sorting algorithms, or graph traversal.

We run this execution time analysis for two functions: chooseSkills and combat. Therefore, when playing the game these two functions are called, we get in return the time of execution of said functions. These analysis are crucial for understanding the performance and efficiency of the application. By examining the runtime of these critical functionalities, the project ensures that the implemented data structures and algorithms are optimized and perform as expected under various conditions, providing insights into potential areas for improvement. (3 hours)

Below we can see an example of the execution time analysis return of one of this functions, especially the chooseSkills one:

```
Time taken to choose skills: 0.000285 seconds
```

Exploratory objectives met

From this part of objectives we were able to implement improvements in the presentation (UX/UI) of the game's interface to enhance the user experience. These enhancements include updated visuals, intuitive navigation, and user-friendly controls, making the game more engaging and accessible. The improvements aim to create a seamless and enjoyable experience for players, ensuring that the interface is both aesthetically pleasing and functional. By focusing on these aspects, the project prioritizes the needs and preferences of the users, leading to a more satisfying and immersive gaming experience. (1 hour)

SOLUTION

System Architecture

1. Data Structures

The game utilizes several custom data structures to represent various elements of the game. These structures include:

- Skill: Represents a skill that a character can use.
- Character: Represents a player-controlled character with attributes (hp,atk,def) and skills.
- Enemy: Represents an enemy character with its own attributes and skills.
- Option: Represents a narrative option that the player can choose during a scenario, which includes responses and enemies.
- Decision: Represents a decision point in a scenario, containing a question and two options.
- Scenario: Represents a game scenario, including a description and a decision point.
- Node: Represents a node in a queue, used for managing the order of scenarios.
- Queue: Represents a queue data structure for handling the sequence of scenarios.

2. Initialization Functions

These functions initialize the game's elements, including skills, characters, and scenarios:

- initializeSkills: Initializes an array of Skill structs with predefined skills.
- initializeCharacters: Initializes an array of Character structs with predefined characters.
- setUpScenarios: Initializes an array of Scenario structs with predefined scenarios.

3. Queue Management

The game uses a queue to manage the sequence of scenarios that the player will encounter:

- initQueue: Initializes the queue.
- isEmpty: Checks if the queue is empty.
- enqueue: Adds a decision to the queue.
- dequeue: Removes and returns a decision from the queue.

4. Character and Skill Selection

Functions for allowing the player to choose a character and their skills:

- chooseCharacter: Prompts the player to choose a character from the list.
- chooseSkills: Allows the player to choose skills for their selected character.

5. Game Scenarios and Decisions

The game is structured around scenarios that include decision points:

- `setupScenarios`: Prepares the scenarios with descriptions and decision points.
- `playGame`: Manages the main game loop, including character and skill selection, scenario management, and combat.

6. Combat System

The combat system handles interactions between the player and enemies:

- `combat`: Simulates a turn-based combat encounter between the player and an enemy.

7. User Interface

Functions to interact with the player:

- `showMenu`: Displays the main menu and starts the game or exits based on the player's choice.

8. Main Function

The main entry point of the game:

- `main`: Calls the `showMenu` function to start the game.

Error Handling

In the project we deal with errors in various cases and we did it in the following way:

1. Input Validation:

- **Character Selection**: When choosing a character, the user's input is validated to ensure it falls within the valid range (1 to 3, which are the number of possible characters). If the input is invalid, the user is prompted to re-enter their choice.
- **Skill Selection**: During skill selection, the input is validated to ensure it corresponds to an available skill (1 to 10) and that the skill has not already been chosen. If the input is invalid, an error message is displayed, and the user is asked to select again.
- **Decision Making**: When making decisions during scenarios, the input is validated to ensure it matches one of the available options (1 or 2). Invalid inputs prompt a re-selection.

2. Queue Operations:

- **Empty Queue Handling**: In the `dequeue` function, if the queue is empty, an error message is displayed, and the program exits. This prevents attempting to dequeue from an empty queue, which would lead to undefined behavior.

3. Combat System:

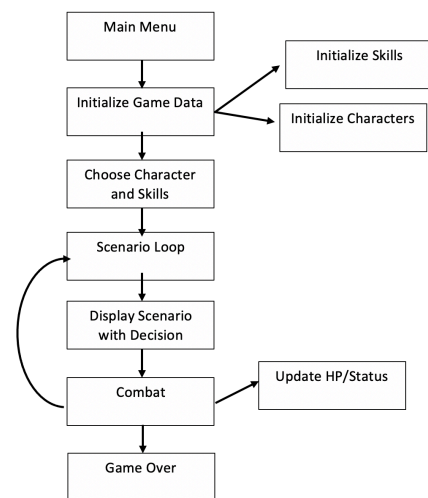
- Skill Usage: In combat, the system checks if the chosen skill has uses left before applying its effects. If the skill has no uses left, an error message is displayed, and the player is prompted to choose another skill.

4. Game Over Condition:

- If the player's health drops to zero during combat, the game ends with a "Game Over" message. This prevents further actions and indicates the end of the game loop since the user has lost the game.

Data model design

The code implements a structured flow where the player navigates through scenarios, makes decisions, and engages in combat, all managed through a series of well-defined data structures and functions.



Dataset description and processing

Our project utilizes multiple datasets to enhance gameplay and provide an engaging user experience. The primary datasets include:

1. Character Attributes Dataset: Contains information on various characters such as health, attack power, defense, and unique skills.
2. Skill Dataset: Details on available skills, including skill names, effects, number of possible uses.
3. Enemy Dataset: Information on enemy types, their attributes, and behaviors.
4. Scenario Dataset: Describes different environments where battles occur.

ETHICAL CONSIDERATIONS [

In the creation process of this game, we have used our previous knowledge on C coding as well as researched online for new techniques and best practices used by other creators to ensure the game's quality and efficiency.

We ensured that all game design, coding, and documentation were original works created by our team. We avoided plagiarism by writing all content from scratch.

When employing Artificial Intelligence tools for idea generation such as skills or code suggestions when we faced a barrier, we ensured that the final implementation was our own work. AI outputs were carefully reviewed and modified to meet the specific requirements of our game.